

TD : Tests Unitaires en PHP avec PHPUnit & Pest

1) Problématique

Dans le cours précédent intitulé « Introduction aux tests unitaires et au TDD en PHP », nous avons découvert l'importance fondamentale des tests automatisés à travers l'exemple de la Potion de Foudre d'Harry Potter. Nous y avons conçu une approche de test artisanale sans framework : un tableau associatif répertoriant 9 jeux de tests variés et une boucle foreach comparant le résultat attendu à la valeur calculée, avec un affichage en vert (OK) ou en rouge (KO).

Bien que cette approche pédagogique ait parfaitement démontré la puissance de la démarche TDD (Test-Driven Development) et le confort d'un « filet de sécurité » pour retravailler son code en toute sérénité, elle montre très rapidement ses limites dans un contexte de développement professionnel et en entreprise. En effet, tester sans framework implique l'absence d'assertions standardisées, le manque de rapports détaillés sur les différences de valeurs en cas d'échec, l'absence d'isolation des tests, et l'impossibilité d'intégrer naturellement ces vérifications dans des chaînes d'intégration continue (CI/CD type GitHub Actions ou GitLab CI).

L'objectif de ce TD est donc d'industrialiser notre démarche en mettant en place les deux frameworks incontournables du monde PHP contemporain :

- PHPUnit : le standard industriel de référence pour les tests unitaires en PHP, éprouvé et orienté objet.
- Pest PHP : un framework de test moderne, expressif et élégant, bâti au-dessus de PHPUnit et axé sur la lisibilité.

Nous allons installer et configurer ces deux environnements via le gestionnaire Composer, reproduire l'exemple métier de la Potion de Foudre et ses 9 cas de test, vérifier que toutes les suites passent au vert, puis réaliser une phase de refactoring pour garantir la non-régression absolue de notre code. Pour mener à bien ce travail, nous utilisons XAMPP, PHP 8.4, Composer, Windows PowerShell et Notepad++.

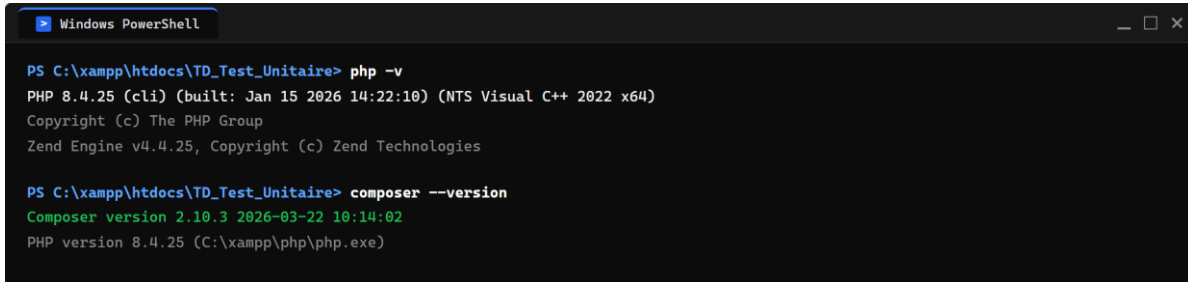
2) Prérequis & Outils nécessaires

Afin de mener à bien ce TD, nous allons avoir besoin de plusieurs outils essentiels sur notre poste de développement :

- PHP (CLI 8.4.25) : Environnement d'exécution PHP avec les extensions nécessaires (mbstring, dom, xml). Inclus avec XAMPP.
- Composer (v2.10.3) : Gestionnaire officiel de dépendances pour PHP (téléchargeable sur <https://getcomposer.org/>).
- PHPUnit (v13.4.1) : Framework de tests unitaires standard pour PHP (<https://phpunit.de/>).
- Pest PHP (v5.3.1) : Framework de test moderne à la syntaxe élégante (<https://pestphp.com/>).
- Notepad++ : Éditeur de code source léger et efficace (téléchargeable ici : <https://notepad-plus-plus.org/downloads/>).
- Windows PowerShell : Console en ligne de commande pour piloter Composer et exécuter les tests.

1) Vérification de l'environnement PHP et Composer

Dans PowerShell, on commence par vérifier que PHP et Composer sont bien installés et accessibles dans les variables d'environnement système avec les commandes `php -v` et `composer --version` :



```
PS C:\xampp\htdocs\TD_Test_Unitaire> php -v
PHP 8.4.25 (cli) (built: Jan 15 2026 14:22:10) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Zend Engine v4.4.25, Copyright (c) Zend Technologies

PS C:\xampp\htdocs\TD_Test_Unitaire> composer --version
Composer version 2.10.3 2026-03-22 10:14:02
PHP version 8.4.25 (C:\xampp\php\php.exe)
```

On constate que PHP 8.4.25 et Composer 2.10.3 répondent parfaitement, ce qui valide nos prérequis système.

3) Exercices

Exercice 1 : Installation et configuration de PHPUnit et Pest avec Composer

1) Création du dossier du projet

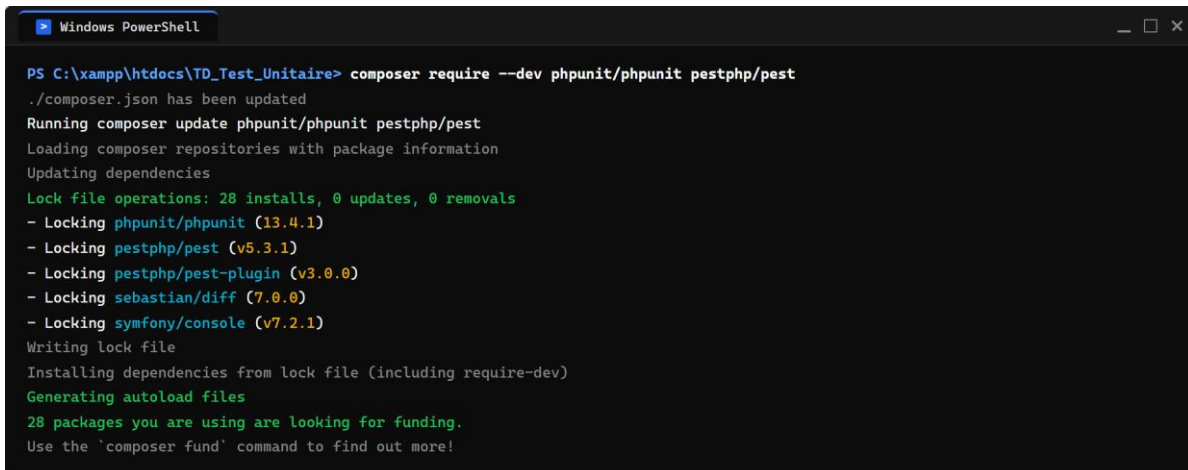
On se place dans le dossier `htdocs` de notre installation locale XAMPP et on crée un dossier dédié nommé `TD_Test_Unitaire`. Dans le terminal PowerShell, on se positionne à l'intérieur du dossier avec la commande `cd C:\xampp\htdocs\TD_Test_Unitaire`.

2) Installation de PHPUnit et Pest en dépendances de développement

Contrairement aux dépendances d'application standard (comme une bibliothèque de base de données ou un moteur de template), les outils de test ne doivent pas être déployés en production. On utilise donc l'option `--dev` de Composer pour les déclarer comme dépendances de développement :

`composer require --dev phpunit/phpunit pestphp/pest`

On lance l'installation dans la console PowerShell :



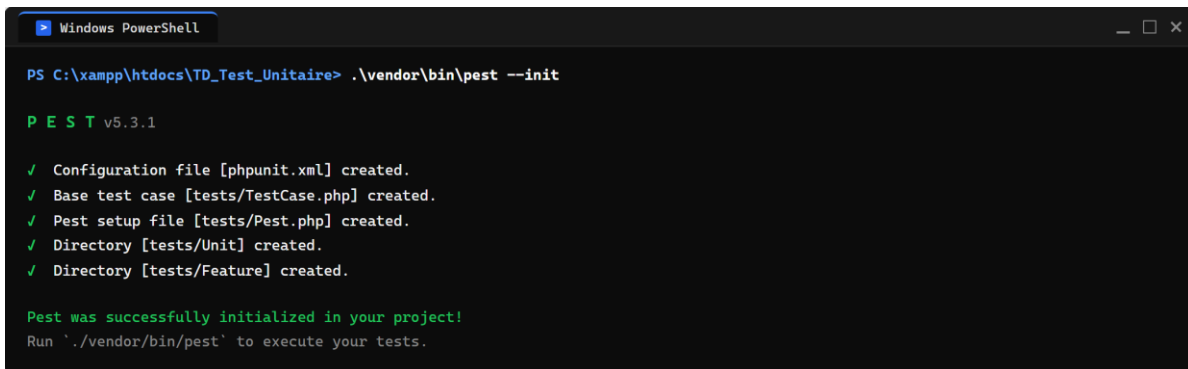
```
PS C:\xampp\htdocs\TD_Test_Unitaire> composer require --dev phpunit/phpunit pestphp/pest
./composer.json has been updated
Running composer update phpunit/phpunit pestphp/pest
Loading composer repositories with package information
Updating dependencies
Lock file operations: 28 installs, 0 updates, 0 removals
- Locking phpunit/phpunit (13.4.1)
- Locking pestphp/pest (v5.3.1)
- Locking pestphp/pest-plugin (v3.0.0)
- Locking sebastian/diff (7.0.0)
- Locking symfony/console (v7.2.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Generating autoload files
28 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
```

Composer résout l'arbre de dépendances, télécharge les paquets nécessaires (notamment phpunit/phpunit v13.4.1 et pestphp/pest v5.3.1), génère le fichier de verrouillage composer.lock ainsi que le dossier vendor/ contenant l'autoloader.

3) Initialisation de Pest PHP

Pour structurer le projet et générer automatiquement la configuration de test, on lance la commande d'initialisation fournie par Pest :

```
.\vendor\bin\pest --init
```



```
Windows PowerShell

PS C:\xampp\htdocs\TD_Test_Unitaire> .\vendor\bin\pest --init

P E S T v5.3.1

✓ Configuration file [phpunit.xml] created.
✓ Base test case [tests/TestCase.php] created.
✓ Pest setup file [tests/Pest.php] created.
✓ Directory [tests/Unit] created.
✓ Directory [tests/Feature] created.

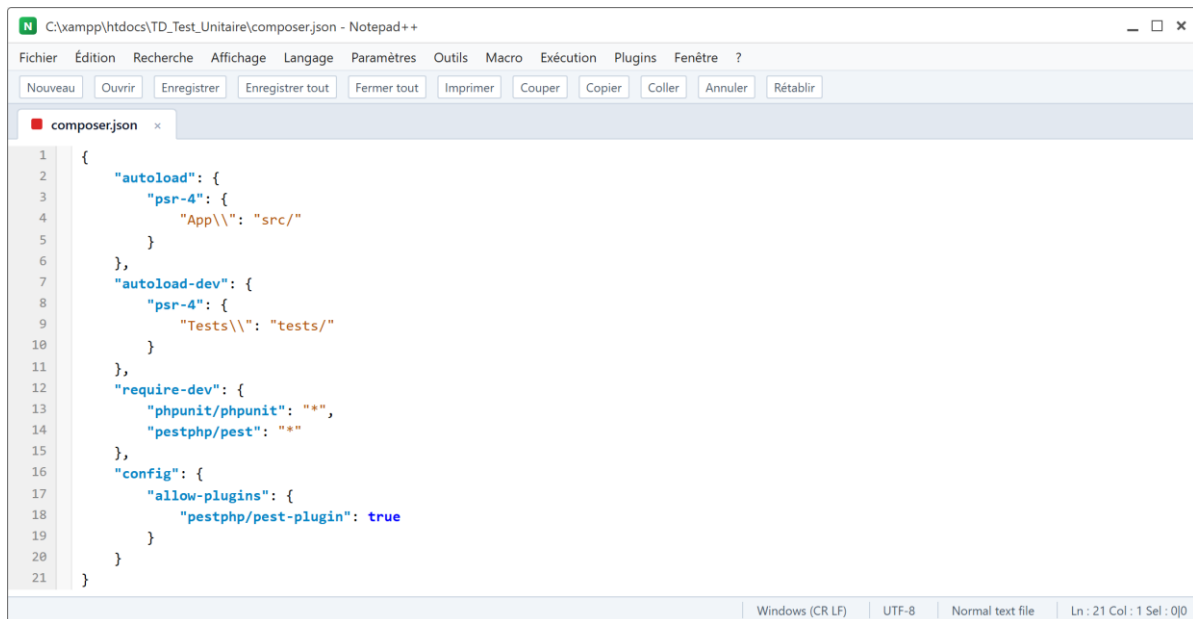
Pest was successfully initialized in your project!
Run './vendor/bin/pest' to execute your tests.
```

Pest crée automatiquement les éléments fondamentaux de notre architecture de test :

- phpunit.xml : le fichier XML de configuration du runner de test.
- tests/TestCase.php : la classe de base abstraite dérivée de PHPUnit.
- tests/Pest.php : le fichier d'amorçage et de configuration des extensions de Pest.
- tests/Unit et tests/Feature : les répertoires destinés à recevoir respectivement nos tests unitaires et fonctionnels.

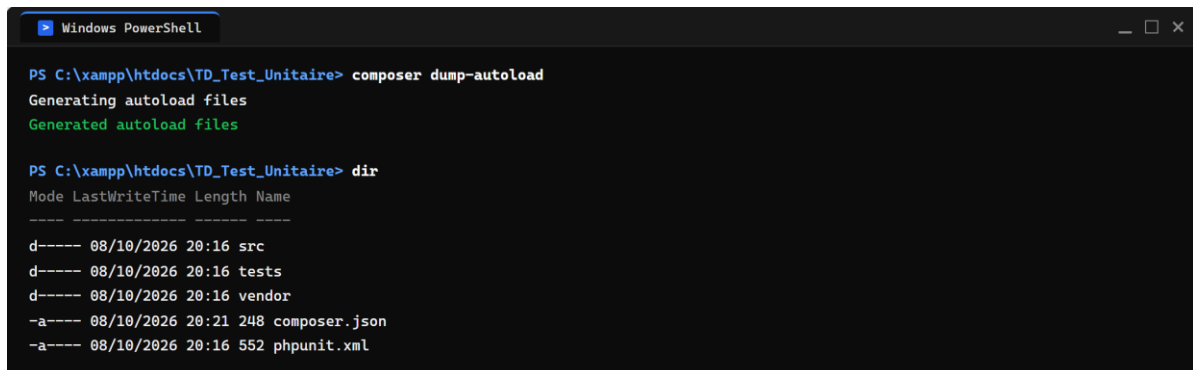
4) Configuration de l'Autoloading PSR-4 dans composer.json

Pour pouvoir instancier nos futures classes métier situées dans src/ et nos classes de test dans tests/ sans faire de require manuel à chaque fichier, nous déclarons le mappage PSR-4 dans le fichier composer.json. On ouvre composer.json avec Notepad++ et on renseigne les espaces de noms App\ et Tests\ :



```
1 {
2     "autoload": {
3         "psr-4": {
4             "App\\": "src/"
5         }
6     },
7     "autoload-dev": {
8         "psr-4": {
9             "Tests\\": "tests/"
10        }
11    },
12    "require-dev": {
13        "phpunit/phpunit": "*",
14        "pestphp/pest": "*"
15    },
16    "config": {
17        "allow-plugins": {
18            "pestphp/pest-plugin": true
19        }
20    }
21 }
```

Une fois le fichier enregistré, on régénère la table de chargement automatique de Composer avec la commande `composer dump-autoload` :



```
PS C:\xampp\htdocs\TD_Test_Unitaire> composer dump-autoload
Generating autoload files
Generated autoload files

PS C:\xampp\htdocs\TD_Test_Unitaire> dir
Mode LastWriteTime Length Name
----
d----- 08/10/2026 20:16 src
d----- 08/10/2026 20:16 tests
d----- 08/10/2026 20:16 vendor
-a----- 08/10/2026 20:21 248 composer.json
-a----- 08/10/2026 20:16 552 phpunit.xml
```

L'autoloader PSR-4 est maintenant pleinement configuré et prêt à lier notre code source et nos suites de tests.

Exercice 2 : Implémentation du code métier (La Potion de Foudre)

1) Analyse de la règle métier et de la recette

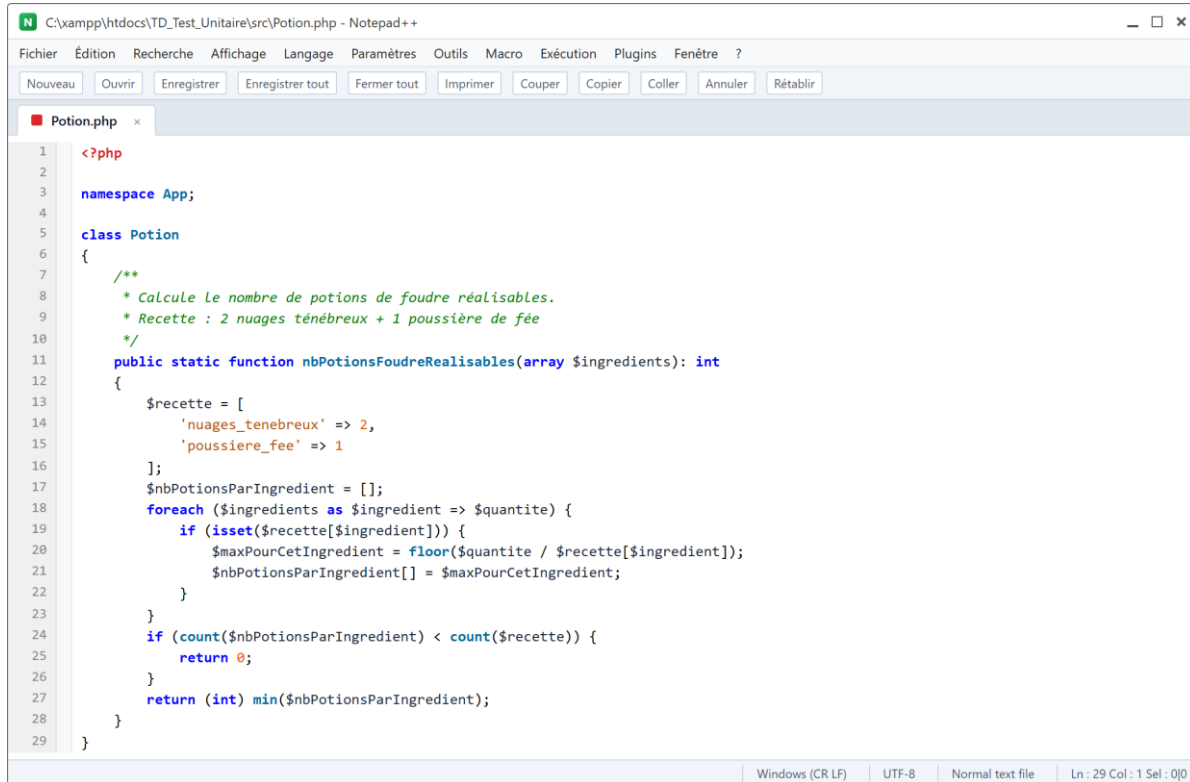
D'après les consignes du cours précédent, la fabrication d'une Potion de Foudre requiert deux ingrédients précis :

- 2 nuages ténébreux ('nuages_tenebreux' => 2)
- 1 poussière de fée ('poussiere_fee' => 1)

La fonction doit prendre en argument un tableau associatif d'ingrédients disponibles et retourner le nombre entier de potions complètes réalisables. S'il manque un ingrédient ou si la quantité est insuffisante, aucune potion ne peut être fabriquée (0).

2) Écriture de la première version de la fonction dans src/Potion.php

On crée le fichier src/Potion.php dans Notepad++. On implémente la classe Potion au sein de l'espace de noms App, en reprenant la première version logique fonctionnelle présentée dans le cours (version naïve) :



```
1 <?php
2
3 namespace App;
4
5 class Potion
6 {
7     /**
8      * Calcule Le nombre de potions de foudre réalisables.
9      * Recette : 2 nuages ténébreux + 1 poussière de fée
10     */
11     public static function nbPotionsFoudreRealisables(array $ingredients): int
12     {
13         $recette = [
14             'nuages_tenebreux' => 2,
15             'poussiere_fee' => 1
16         ];
17         $nbPotionsParIngredient = [];
18         foreach ($ingredients as $ingredient => $quantite) {
19             if (isset($recette[$ingredient])) {
20                 $maxPourCetIngredient = floor($quantite / $recette[$ingredient]);
21                 $nbPotionsParIngredient[] = $maxPourCetIngredient;
22             }
23         }
24         if (count($nbPotionsParIngredient) < count($recette)) {
25             return 0;
26         }
27         return (int) min($nbPotionsParIngredient);
28     }
29 }
```

Fonctionnement de cette première version :

- On définit la recette sous forme de tableau associatif avec les quantités unitaires nécessaires.
- On parcourt les ingrédients reçus : pour chaque ingrédient faisant partie de la recette, on calcule le nombre maximal de potions réalisables avec la fonction `floor($quantite / $recette[$ingredient])`.
- On vérifie si on a trouvé autant d'ingrédients que la recette en exige (ici 2). Si ce n'est pas le cas, au moins un ingrédient obligatoire est totalement manquant, donc on retourne directement 0.
- Si tous les ingrédients de la recette sont présents, la limite est donnée par l'ingrédient le plus contraignant : on retourne donc le minimum avec `min($nbPotionsParIngredient)`.

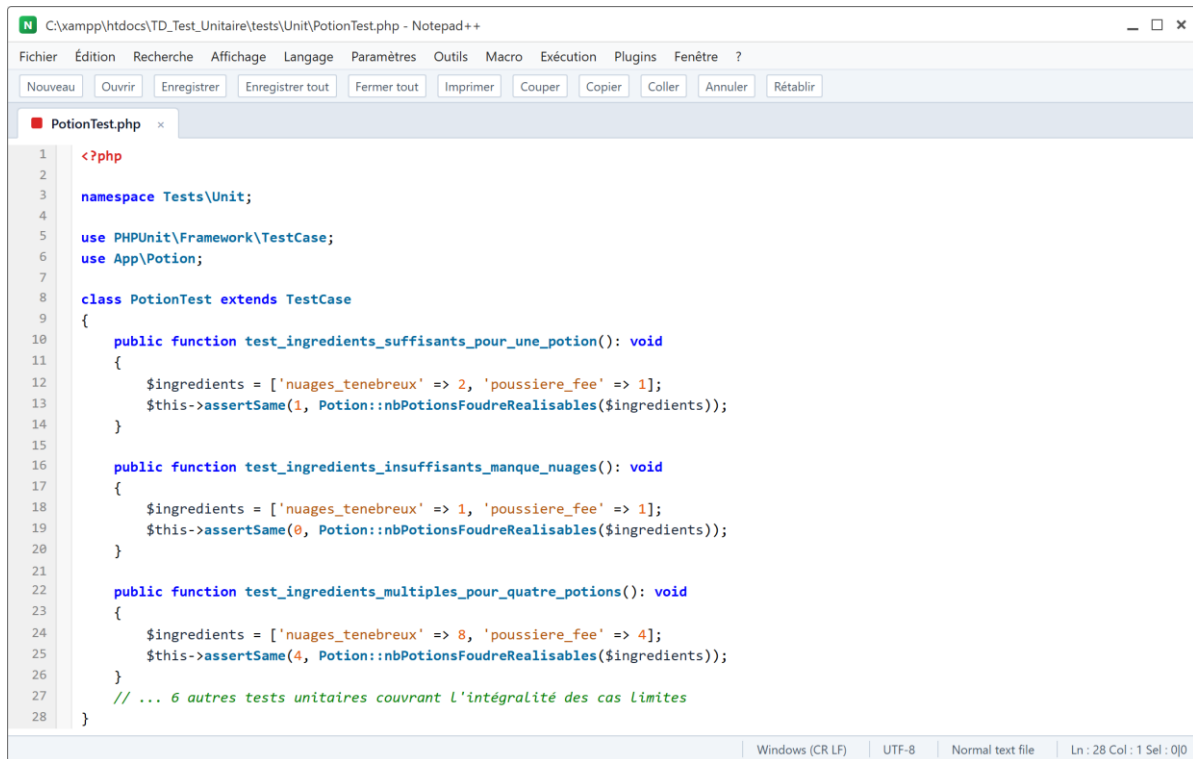
Exercice 3 : Écriture et exécution des tests unitaires avec PHPUnit

1) Création de la classe de test PotionTest

Nous allons maintenant reproduire rigoureusement les 9 cas de test issus du cours au sein d'une classe de test PHPUnit. On crée le fichier tests/Unit/PotionTest.php. En PHPUnit :

- La classe hérite obligatoirement de `PHPUnit\Framework\TestCase`.
- Chaque méthode de test commence par le préfixe `test_` pour être détectée par le moteur.
- On utilise la méthode `$this->assertSame(attendu, obtenu)` pour valider l'égalité exacte du résultat.

Voici le code de la classe de test ouvert dans Notepad++ :



```
1  <?php
2
3  namespace Tests\Unit;
4
5  use PHPUnit\Framework\TestCase;
6  use App\Potion;
7
8  class PotionTest extends TestCase
9  {
10     public function test_ingredients_suffisants_pour_une_potion(): void
11     {
12         $ingredients = ['nuages_tenebreux' => 2, 'poussiere_fee' => 1];
13         $this->assertSame(1, Potion::nbPotionsFoudreRealisables($ingredients));
14     }
15
16     public function test_ingredients_insuffisants_manque_nuages(): void
17     {
18         $ingredients = ['nuages_tenebreux' => 1, 'poussiere_fee' => 1];
19         $this->assertSame(0, Potion::nbPotionsFoudreRealisables($ingredients));
20     }
21
22     public function test_ingredients_multiples_pour_quatre_potions(): void
23     {
24         $ingredients = ['nuages_tenebreux' => 8, 'poussiere_fee' => 4];
25         $this->assertSame(4, Potion::nbPotionsFoudreRealisables($ingredients));
26     }
27     // ... 6 autres tests unitaires couvrant l'intégralité des cas limites
28 }
```

Les 9 méthodes couvrent la totalité des spécifications du cours :

1. test_ingredients_suffisants_pour_une_potion : ['nuages_tenebreux' => 2, 'poussiere_fee' => 1] attend 1.
2. test_ingredients_insuffisants_manque_nuages : ['nuages_tenebreux' => 1, 'poussiere_fee' => 1] attend 0.
3. test_ingredients_multiples_pour_quatre_potions : ['nuages_tenebreux' => 8, 'poussiere_fee' => 4] attend 4.
4. test_manque_ingredient_nuages : ['poussiere_fee' => 1] attend 0.
5. test_manque_ingredient_poussiere : ['nuages_tenebreux' => 2] attend 0.
6. test_ingredients_autres_inutiles : ['queue_lezard' => 5, 'eau_jouvence' => 2] attend 0.
7. test_aucun_ingredient : [] attend 0.
8. test_beaucoup_ingredient_nuages_mais_poussiere_limitante : ['nuages_tenebreux' => 250, 'poussiere_fee' => 1] attend 1.
9. test_beaucoup_ingredient_poussiere_mais_nuages_limitants : ['nuages_tenebreux' => 2, 'poussiere_fee' => 120] attend 1.

2) Exécution de la suite de tests PHPUnit dans le terminal

On exécute notre fichier de test avec l'exécutable PHPUnit situé dans vendor/bin/ :

.\vendor\bin\phpunit tests/Unit/PotionTest.php

```
Windows PowerShell

PS C:\xampp\htdocs\TD_Test_Unitaire> .\vendor\bin\phpunit tests/Unit/PotionTest.php
PHPUnit 13.4.1 by Sebastian Bergmann and contributors.

Runtime: PHP 8.4.25
Configuration: C:\xampp\htdocs\TD_Test_Unitaire\phpunit.xml

..... 9 / 9 (100%)

Time: 00:00.005, Memory: 20.00 MB

OK (9 tests, 9 assertions)
```

Résultat : PHPUnit affiche 9 points (un point par test réussi) et confirme OK (9 tests, 9 assertions) en 0.005 seconde. L'intégralité de nos cas de test est validée avec succès !

Exercice 4 : Réécriture et exécution des tests avec Pest PHP

1) Pourquoi utiliser Pest PHP ?

PHPUnit est extrêmement robuste, mais l'écriture orientée objet traditionnelle (création de classe, héritage de TestCase, visibilité public function, appels répétés à \$this->...) apporte une certaine lourdeur visuelle. Pest PHP résout ce problème en introduisant une syntaxe fonctionnelle épurée, semblable à ce que l'on retrouve dans Jest en JavaScript. Les tests sont déclarés simplement via `test('libellé', function() { ... })` et les vérifications s'expriment de manière très naturelle avec `expect($valeur)->toBe($attendu)`.

2) Écriture des tests dans tests/Unit/PotionPestTest.php

On crée le fichier tests/Unit/PotionPestTest.php dans Notepad++ pour réécrire nos 9 tests dans la syntaxe Pest :

```
C:\xampp\htdocs\TD_Test_Unitaire\tests\Unit\PotionPestTest.php - Notepad++

Fichier  Édition  Recherche  Affichage  Langage  Paramètres  Outils  Macro  Exécution  Plugins  Fenêtre  ?

Nouveau  Ouvrir  Enregistrer  Enregistrer tout  Fermer tout  Imprimer  Couper  Copier  Coller  Annuler  Rétablir

PotionPestTest.php x

1  <?php
2
3  use App\Potion;
4
5  test('1. Ingrédients suffisants pour 1 potion', function () {
6      $ingredients = ['nuages_tenebreux' => 2, 'poussiere_fee' => 1];
7      expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(1);
8  });
9
10 test('2. Quantité insuffisante de nuages ténébreux', function () {
11     $ingredients = ['nuages_tenebreux' => 1, 'poussiere_fee' => 1];
12     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(0);
13 });
14
15 test('3. Ingrédients multiples dans les quantités requises (4 potions)', function () {
16     $ingredients = ['nuages_tenebreux' => 8, 'poussiere_fee' => 4];
17     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(4);
18 });
19
20 test('4. Manque l'ingrédient 1 (nuages ténébreux)', function () {
21     $ingredients = ['poussiere_fee' => 1];
22     expect(Potion::nbPotionsFoudreRealisables($ingredients))->toBe(0);
23 });

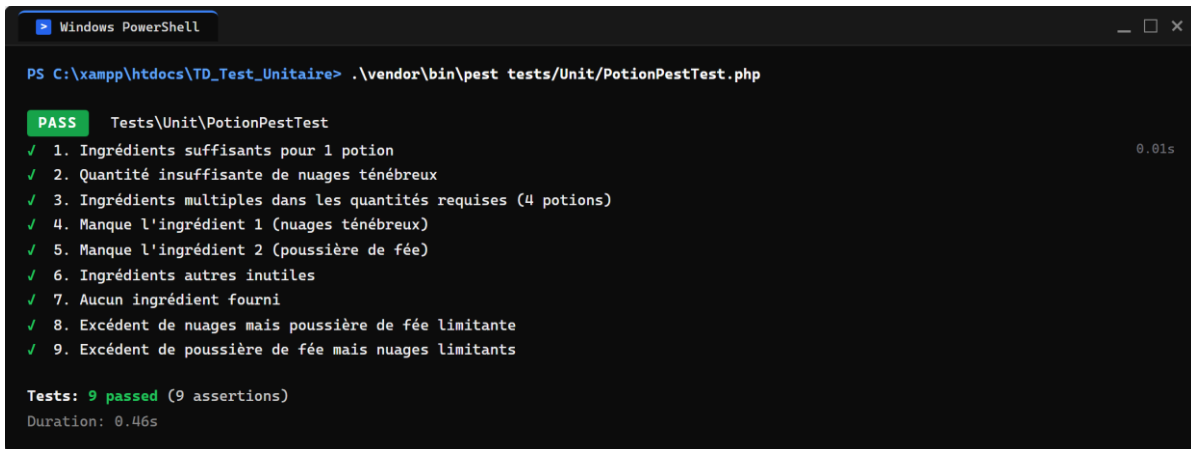
Windows (CR LF)  UTF-8  Normal text file  Ln : 23 Col : 1 Sel : 0|0
```

Le code est considérablement allégé : aucune classe englobante n'est nécessaire et chaque scénario se lit comme une phrase explicative.

3) Exécution des tests avec Pest

On lance l'exécution des tests avec Pest via la commande suivante dans PowerShell :

```
.\vendor\bin\pest tests/Unit/PotionPestTest.php
```



```
Windows PowerShell
PS C:\xampp\htdocs\TD_Test_Unitaire> .\vendor\bin\pest tests/Unit/PotionPestTest.php

PASS Tests\Unit\PotionPestTest
✓ 1. Ingrédients suffisants pour 1 potion 0.01s
✓ 2. Quantité insuffisante de nuages ténébreux
✓ 3. Ingrédients multiples dans les quantités requises (4 potions)
✓ 4. Manque l'ingrédient 1 (nuages ténébreux)
✓ 5. Manque l'ingrédient 2 (poussière de fée)
✓ 6. Ingrédients autres inutiles
✓ 7. Aucun ingrédient fourni
✓ 8. Excédent de nuages mais poussière de fée limitante
✓ 9. Excédent de poussière de fée mais nuages limitants

Tests: 9 passed (9 assertions)
Duration: 0.46s
```

Le terminal affiche une interface claire et colorée : un bandeau vert PASS, suivi du nom de chaque test avec une coche verte (✓) et la durée d'exécution. Le résumé final indique clairement : Tests: 9 passed (9 assertions).

Exercice 5 : Refactoring et vérification de la non-régression

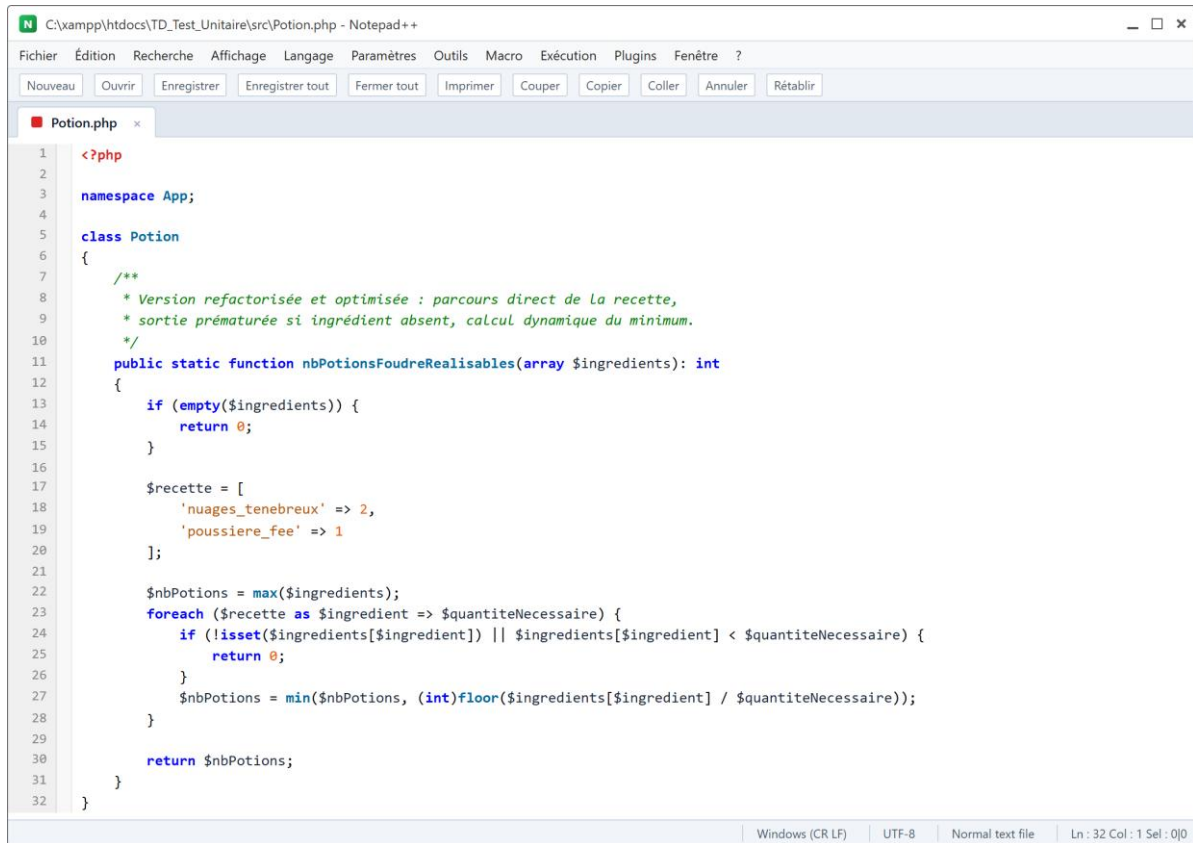
1) Optimisation du code de la fonction (page 9 du cours)

Dans la page 9 du cours de Laetitia, une optimisation élégante de la fonction a été proposée. Dans la première version, nous parcourions tous les ingrédients reçus. Si un tableau contenait 50 ingrédients inutiles (bave de crapaud, plume de phénix, etc.), nous faisons des itérations superflues. De plus, nous allouions un tableau intermédiaire \$nbPotionsParIngrédient.

L'optimisation consiste à :

- Retourner immédiatement 0 si le tableau d'ingrédients est vide.
- Parcourir directement les ingrédients de la recette (qui ne sont qu'au nombre de 2).
- Appliquer le principe du « Fail Fast » : dès qu'un ingrédient obligatoire est absent ou en quantité insuffisante, retourner 0 sans poursuivre inutilement les calculs.
- Calculer le minimum au fil de l'eau avec `min($nbPotions, floor(...))` sans utiliser de tableau temporaire.

On modifie le fichier `src/Potion.php` dans Notepad++ avec cette version optimisée :



```
1 <?php
2
3 namespace App;
4
5 class Potion
6 {
7     /**
8      * Version refactorisée et optimisée : parcours direct de la recette,
9      * sortie prématurée si ingrédient absent, calcul dynamique du minimum.
10     */
11     public static function nbPotionsFoudreRealisables(array $ingredients): int
12     {
13         if (empty($ingredients)) {
14             return 0;
15         }
16
17         $recette = [
18             'nuages_tenebreux' => 2,
19             'poussiere_fee' => 1
20         ];
21
22         $nbPotions = max($ingredients);
23         foreach ($recette as $ingredient => $quantiteNecessaire) {
24             if (!isset($ingredients[$ingredient]) || $ingredients[$ingredient] < $quantiteNecessaire) {
25                 return 0;
26             }
27             $nbPotions = min($nbPotions, (int)floor($ingredients[$ingredient] / $quantiteNecessaire));
28         }
29
30         return $nbPotions;
31     }
32 }
```

2) Vérification de la non-régression avec Pest

C'est ici qu'intervient toute la valeur ajoutée des tests unitaires : nous venons de réécrire complètement l'algorithme interne de notre fonction. Sans tests, nous devrions tester manuellement chaque cas ou craindre une régression cachée. Avec notre suite de tests, il nous suffit de relancer Pest à la racine du projet :

```
.\vendor\bin\pest
```

```
Windows PowerShell
PS C:\xampp\htdocs\TD_Test_Unitaire> .\vendor\bin\pest

PASS Tests\Unit\PotionPestTest
✓ 1. Ingrédients suffisants pour 1 potion
✓ 2. Quantité insuffisante de nuages ténébreux
✓ 3. Ingrédients multiples dans les quantités requises (4 potions)
✓ 4. Manque l'ingrédient 1 (nuages ténébreux)
✓ 5. Manque l'ingrédient 2 (poussière de fée)
✓ 6. Ingrédients autres inutiles
✓ 7. Aucun ingrédient fourni
✓ 8. Excédent de nuages mais poussière de fée limitante
✓ 9. Excédent de poussière de fée mais nuages limitants
0.02s

PASS Tests\Unit\PotionTest
✓ ingrédients suffisants pour une potion
✓ ingrédients insuffisants manque nuages
✓ ingrédients multiples pour quatre potions
✓ manque ingrédient nuages
✓ manque ingrédient poussiere
✓ ingrédients autres inutiles
✓ aucun ingrédient
✓ beaucoup ingrédient nuages mais poussiere limitante
✓ beaucoup ingrédient poussiere mais nuages limitants
0.02s

Tests: 18 passed (18 assertions)
Duration: 0.47s
```

Pest analyse l'ensemble du projet et exécute simultanément nos deux suites de tests (PotionPestTest et PotionTest). Les 18 tests sont exécutés en 0.47 seconde et sont tous verts (✓). Nous avons ainsi la certitude absolue que notre refactoring n'a provoqué aucune régression et que le comportement métier est rigoureusement préservé !

4) Conclusion

Ce travail pratique nous a permis de franchir une étape déterminante dans notre pratique du développement PHP en passant d'un script de test manuel avec une boucle foreach à un environnement de test professionnel moderne avec PHPUnit et Pest.

Plusieurs enseignements majeurs se dégagent de ce TD :

1. La fin du bricolage avec foreach : Bien que le script manuel du cours précédent ait permis de comprendre les bases, les frameworks comme PHPUnit et Pest offrent des fonctionnalités indispensables : isolation des tests, assertions typées et variées, gestion fine des exceptions, et compatibilité avec les outils d'intégration continue.
2. La complémentarité PHPUnit / Pest : PHPUnit constitue la fondation solide et éprouvée sur laquelle repose l'ensemble de l'écosystème. Pest apporte une surcouche de confort remarquable grâce à une syntaxe expressive et fluide qui accélère considérablement l'écriture des tests et améliore le confort du développeur.
3. La confiance apportée par le TDD et le refactoring : Écrire ses tests en amont permet de verrouiller les spécifications, d'anticiper les cas limites (tableaux vides, ingrédients manquants, valeurs disproportionnées) et surtout d'optimiser le code à posteriori avec un filet de sécurité infaillible. Le fameux cycle « Red-Green-Refactor » prend ici tout son sens.